

Inventario Modular

Tareas, visor y Android LAN

Documento de continuidad tecnica
y puesta en marcha sin Internet

Entrega: 10 de septiembre de 2026
Rama: primeros-pasos

ESTADO DE LA ENTREGA

Servidor y web verificados localmente. APK piloto compilada. Pendientes: firma release, HTTPS institucional y prueba en telefonos reales.

No se despliego en produccion. Este documento explica como preparar, validar, instalar y revertir la entrega sin servicios externos.

Contenido

Traspaso tecnico: tareas, visor y Android LAN	3
1. Resumen y estado real	3
2. Requerimientos y decisiones	3
3. Recorrido funcional	4
4. Contratos y persistencia	4
5. Android y seguridad de red	5
6. Mapa del codigo y pruebas	5
7. Pendientes y riesgos conocidos	6
8. Continuar con otra IA	7
9. Referencias	7
 Instalacion en el trabajo sin Internet	 8
1. Que llevar y que se puede habilitar	8
2. Inventario previo y condiciones de parada	8
3. Transferencia y preparacion	9
4. Respaldo antes del primer arranque nuevo	9
5. Entorno, base y publicacion del JAR	10
6. HTTPS institucional sin afectar otros sistemas	11
7. Instalar Android y validar el circuito	12
8. Firma release en una estacion preparada	12
9. Aceptacion, problemas y reversion	13

Traspaso tecnico: tareas, visor y Android LAN

Entrega documental del 10 de septiembre de 2026. Proyecto Inventario Modular, Centro Judicial San Pedro. Repositorio: <https://gitlab.com/gustavoeliasm/inventario-modular> Rama: primeros-pasos. Base anterior a esta entrega: f4d5960.

Este documento y `instalacion-tareas-lan-2026-09-10.md` son las fuentes editables del PDF `output/pdf/traspaso-tareas-lan.pdf`. El código del commit de entrega prevalece sobre documentos históricos. El commit exacto queda en `VERSION.txt` del paquete transportable; no se incluye en este PDF para evitar una referencia circular al commit.

1. Resumen y estado real

El administrador dispone de un visor independiente para PC/TV y de accesos desde el Panel General. Los técnicos disponen de una web móvil y una APK Android piloto para operar tareas y consultar avisos dentro de la LAN. El servidor de mensajes es el mismo Spring Boot: no se agregaron Firebase, FCM, servicios Google, MQTT ni otro demonio Linux.

La web y las API están implementadas y probadas localmente. La APK se compiló y se verificó su firma debug. NO se probó un teléfono físico, Active Directory real ni el MySQL del trabajo durante esta entrega. No se desplegó en producción.

La distribución general de la APK queda condicionada a HTTPS institucional, firma release, validación de batería/Doze y prueba de campo. El piloto no es una garantía de recepción permanente. La aplicación no funciona para editar tareas sin conexión.

2. Requerimientos y decisiones

1. Ver tareas en una dirección independiente, sin menú general, para la sala de reuniones.
2. Crear tareas, ver comentarios, identificar solicitante y técnico responsable.
3. Mostrar pendientes y actividad diaria, con búsqueda y fichas compactas.
4. Permitir acceso local o de dominio desde celulares conectados por Wi-Fi institucional.
5. Avisar con sonido cuando otra persona crea una tarea, sin servidores externos.
6. Mantener autenticación, permisos del módulo, auditoría y protección CSRF existentes.

Se conservaron los estados de base PENDIENTE, EN_PROCESO, CERRADA y CANCELADA. Tomar una tarea asigna responsable y utiliza EN_PROCESO; la interfaz móvil y la etiqueta del visor presentan ambas situaciones abiertas como Pendiente. No se hizo una migración destructiva del enum. Consultar las inconsistencias pendientes en la sección 7.

La web abierta usa sonido WebAudio activado por el usuario. No se eligió una PWA como solución de avisos en reposo: el navegador puede suspenderla. Android usa un servicio visible durante una jornada activada por el técnico, con consulta periódica al servidor. Las restricciones de reposo siguen aplicando y requieren prueba por modelo de teléfono.

3. Recorrido funcional

Visor: /admin/tareas/visor. Es una plantilla autonoma sin barra lateral ni cabecera del inventario. Ofrece filtros, formularios y comentarios, conservando el origen del formulario mediante `origen=visor`. Las fichas se compactaron para mostrar mas tareas. El modo En vivo refresca el tablero y contadores cada 30 segundos. Si se escribe en un formulario o este tiene el foco, se pausa para preservar el borrador. Guardar o volver a abrir la pagina permite reanudar. No hay modo publico sin autenticacion.

Movil: /movil/login y /movil/tareas. El formulario envia a /login con `destino=movil`, y vuelve al modulo movil. Un destino diferente no puede redirigir a una web externa. La cuenta debe tener TAREAS/VER; las acciones requieren los permisos y reglas de propiedad de la API existente.

El movil permite crear, tomar, editar, comentar, cerrar, cancelar, reabrir y eliminar segun autorizacion. Un tecnico crea tareas a su nombre; el administrador puede dejarlas sin tomar. Hay vistas Pendientes, Mis tareas, Finalizadas y Todas. La busqueda movil incluye ID, titulo, descripcion, solicitante, fuero, responsable y equipo; no comentarios. La busqueda del visor si incluye comentarios. La nueva tarea movil usa PC-GENERICA; no hay selector de equipo en ese formulario. Al editar se conserva el equipo existente.

Se muestran hasta 40 filas por tanda con Ver mas. Esto limita el renderizado, no el volumen de la respuesta de la API: la paginacion real del servidor sigue pendiente.

4. Contratos y persistencia

Las nuevas rutas de solo lectura son:

- GET /api/v1/movil/sesion: usuario actual, puedeEditar y administrador.
- GET /api/v1/movil/avisos: cursor actual, sin historial, para iniciar seguimiento.
- GET /api/v1/movil/avisos?despuesDe=N: hasta 100 eventos posteriores, en orden.
- GET /api/v1/movil/apk: descarga autenticada de un archivo configurado en el servidor.

Las mutaciones reutilizan /api/v1/tareas-tecnicas, incluidos tomar, estado y comentarios. No se creo una segunda logica de negocio para celulares. Las API sin sesion responden 401; sin permiso, 403; tomar una tarea finalizada o asignada a otro tecnico produce 409. Un cursor negativo produce 400. La APK ausente produce 404. CSRF no se desactivo.

Ejemplo de respuesta de avisos (datos ficticios):

```
{
  "siguiente": 42,
  "avisos": [{
    "id": 42,
    "tareaId": 18,
    "titulo": "Revisar impresora",
    "autor": "administrador.ejemplo",
    "creadoEn": "2026-09-10T09:00:00"
  }]
}
```

V15 crea `tareas_avisos_secuencia`, con una fila `id=1` y `ultimo_id`, y `tareas_avisos`, con `id`, `tarea_id`, `titulo`, `autor` y `creado_en`. Son tablas aditivas; no borran tareas. La secuencia usa `SELECT FOR UPDATE` dentro de la misma transaccion que la creacion, con propagacion `MANDATORY`. La tarea, auditoria y aviso confirman o revierten juntos. No reemplazar la secuencia por un autoincremento sin analizar el orden de confirmacion: un cursor podria adelantarse y omitir una transaccion que confirma mas tarde.

El cursor de respuesta avanza hasta el ultimo evento entregado, no hasta el ultimo existente si quedan otras paginas. Ante una restauracion con cursor cliente mayor al servidor, se vuelve al punto actual sin emitir

historial. No se garantiza entrega exactamente una vez: un cierre entre notificación y persistencia puede repetir un aviso.

No existe FK de aviso a tarea: una eliminación no borra el anuncio histórico. No hay purga, destinatarios por sede, acuses de lectura ni escalamiento. Los anuncios son visibles para todas las cuentas con TAREAS/VER. Solo la creación genera un aviso; comentarios, toma y cierre no generan sonido en esta versión.

Dos tomas simultáneas usan el bloqueo pesimista `buscarParaTomar` y verifican asignación y estado bajo ese bloqueo. Esto protege la toma; no constituye una auditoría completa de todas las carreras posibles entre editar, cerrar, eliminar y reasignar.

5. Android y seguridad de red

Paquete release: `ar.gov.justiciajujuy.tareaslan`. Piloto: el mismo con sufijo `.piloto`. Version 0.1.0; `versionCode` 1. Mínimo Android 8 / API 26; compile y target SDK 35. WebView requiere Chromium 103 o superior para las API JavaScript utilizadas. Gradle 8.11.1, Android Gradle Plugin 8.9.2, Build Tools 35.0.0, Java 17 o 21.

`MainActivity` aloja el WebView con navegación al mismo origen, sin puente JavaScript nativo, sin acceso a archivos y sin cookies de terceros. La app usa recursos locales; Safe Browsing está desactivado para no realizar consultas externas desde ese mecanismo. La sesión de Spring se comparte mediante `CookieManager`. No se persiste la clave en código nativo; la cookie de sesión sigue siendo sensible y requiere custodia del celular.

`LanClient` admite un origen sin ruta ni credenciales. Release solo admite HTTPS; debug también permite HTTP para laboratorio. Se verifican IP privadas/loopback, origen y TLS. La CA institucional instalada puede ser usada por la APK; no se ignoran errores de certificado ni de nombre. La validación de IP no reemplaza un firewall de salida ni constituye una defensa completa ante manipulación de DNS.

`AvisosService` se activa con Avisos. Usa notificación permanente, canal de tareas con sonido/vibración y consulta cada 10 segundos; sin red aplica espera hasta 60. Cada ciclo verifica la identidad activa. Un lote lleno se continúa al segundo siguiente. El cursor se guarda por origen/usuario después de notificar. No suenan eventos propios. La primera activación establece el punto actual, sin hacer sonar tareas anteriores.

Mantiene un bloqueo parcial de CPU mientras está activo: consume batería y no elimina todas las restricciones de Doze. Al detenerlo libera el bloqueo. No arranca al reiniciar el teléfono ni tras forzar cierre; usa `START_NOT_STICKY`. Una sesión vencida o permiso revocado obliga a ingresar y activar de nuevo. No molestar, volumen, canal silenciado, restricciones del fabricante y pérdida de Wi-Fi pueden impedir el aviso.

El permiso Android INTERNET también permite acceso LAN; no implica dependencia de Internet. Para cumplir el aislamiento total, aplicar reglas institucionales en Wi-Fi, VLAN y firewall. La descarga de dependencias se hace solo al preparar la compilación, no al ejecutar el JAR o la APK. No hay implementación equivalente para iPhone.

6. Mapa del código y pruebas

Base Java: `src/main/java/ar/gov/justiciajujuy/sanpedro/inventario/`.

- `web/TareaTecnicaPageController.java`: visor, búsquedas, métricas y retorno de formularios.
- `web/TareaMovilController.java`: página móvil, sesión, avisos y APK protegida.
- `web/TareaTecnicaController.java`: API existente y reglas de autorización de operaciones.
- `tareas/TareaTecnicaService.java`: creación transaccional, toma y resumen diario.

- tareas/TareaAvisoService.java: secuencia persistente, lotes y cursores.
- tareas/TareaTecnicaRepository.java: busqueda ampliada y bloqueo de toma.
- config/SecurityConfig.java: login y destinos fijos, seguridad de API y filtro LAN.
- config/DataSourceConfig.java: seleccion primaria/fallback del perfil local.

Recursos: src/main/resources/.

- templates/admin/tareas-visor.html, static/css/admin.css: visor compacto independiente.
- templates/movil/, static/css/movil.css, static/js/movil-tareas.js: interfaz movil.
- static/js/visor-live.js: refresco sin perder formularios.
- static/js/lucide.min.js y lucide.LICENSE: iconos locales, version 0.468.0.
- db/migration/V15__avisos_tareas_lan.sql: tablas de avisos.
- application-{local,casa}.properties: inicializacion SQL; Flyway sigue apagado.

Android: android/app/src/main/java/ar/gov/justiciajujuy/tareaslan/ contiene MainActivity, LanClient y AvisosService. Revisar tambien Manifest, reglas de respaldo y configuraciones TLS main/debug. android/README.md contiene compilacion y prueba.

Pruebas: src/test/java/.../web/TareaMovilControllerTests.java tiene siete casos que cubren login/destino seguro, permisos/CSRF, avisos, rollback, paginacion, inicializacion idempotente, restauracion de cursor y toma concurrente/finalizada. Los ocho casos de TareaTecnicaPageControllerTests cubren las paginas administrativas.

Evidencia local registrada en esta entrega:

- Suite Maven: 125 pruebas, cero fallos, errores y omitidas.
- APK debug: compilacion y lint correctos; firma verificada con apksigner.
- Lint Android: cero errores; advertencias pendientes documentadas en android/README.md.
- Navegador: recorrido crear, tomar, comentar y cerrar; escritorio y movil de 390 px sin desborde.
- Visor: refresco automatico observado, consola sin errores durante la prueba.
- Descarga autenticada de APK: HTTP 200, tipo correcto y hash igual al archivo.

La tarea local #35 fue una prueba del circuito y quedo cerrada. La base H2 de casa y sus credenciales no se distribuyen. Los reportes Maven se generan en target/surefire-reports; el kit incluye un resumen de resultados y hashes. La CI de GitLab ejecuta Maven y publica JAR por siete dias; no compila Android ni genera este PDF. Un push exitoso no demuestra que el runner haya ejecutado o aprobado la pipeline.

7. Pendientes y riesgos conocidos

Prioridad antes de anunciar produccion movil:

1. Firmar release con clave institucional custodiada fuera del repositorio, habilitar HTTPS confiable y probar en telefonos reales. El kit trae solo APK piloto debug.
2. Ensayar V15 y arranque con una copia MySQL del trabajo. Las pruebas actuales usan H2.
3. Verificar AD real, roles locales, Wi-Fi, DNS, certificado, perdida de red y reposo.
4. Resolver semantica de metricas del visor: realizadasHoy usa cerradoEn e incluye canceladas; Between incluye el limite de medianoche del dia siguiente. Usar estado CERRADA e intervalo semiabierto, con pruebas de cancelacion y cambio de dia.

5. Unificar filtros: el contador Pendientes suma PENDIENTE y EN_PROCESO, pero el filtro PENDIENTE selecciona solo ese enum. Los selectores aun muestran EN_PROCESO. El movil ya agrupa ambos. Algunos botones Tomar administrativos pueden aparecer en tareas finalizadas sin responsable; el backend ahora rechaza esa accion con 409.

Prioridad posterior:

- Agregar paginacion real, medir volumen de comentarios y carga LDAP: el refresco del visor solicita la pagina completa y puede recalculer datos del dominio cada 30 segundos.
- Acordar retencion de avisos sin destruir cursores activos; no hay limpieza automatica.
- Evaluar acuses por dispositivo y alcance por sede si se requiere trazabilidad de entrega.
- Revisar bateria y permisos por fabricante, pruebas instrumentadas y distribucion interna.
- Probar conflictos entre cierre/edicion/toma; el test actual cubre dos tomas simultaneas.
- Unificar migraciones antes de activar Flyway: el historial contiene versiones V6 duplicadas.
- Revisar fallback MySQL y confianza de cabeceras proxy. LAN-only no es aislamiento completo.

No resolver estos puntos borrando estados, desactivando CSRF/TLS o agregando push externo.

8. Continuar con otra IA

Entregar este PDF, ambas fuentes Markdown y el repositorio o bundle de la entrega. No entregar archivos de entorno, claves de firma, cookies, respaldos de base ni credenciales de dominio. Graphify contiene un grafo local anterior al modulo movil: sirve como mapa historico, pero no como fuente completa de esta version; esta ignorado por Git.

Prompt sugerido:

```
Continua Inventario Modular, rama primeros-pasos.
Lee docs/inventario-modular/traspaso-tareas-lan-2026-09-10.md
y docs/inventario-modular/instalacion-tareas-lan-2026-09-10.md.
Verifica git status y el commit del paquete antes de editar.
No reviertas cambios ajenos ni uses servicios externos en ejecucion.
El servidor es Java 21 / Spring Boot 4.0.1 / MySQL.
La APK Android es piloto: no afirmar que funciona en reposo sin medirlo.
Prioriza la lista de pendientes y conserva permisos, CSRF y TLS.
No modificar el servicio legado inventario.service.
Actualiza pruebas, documentacion y bitacora con cada cambio.
```

9. Referencias

Fuentes primarias consultadas para firma y restricciones Android el 10/09/2026:

- Firma y compilacion por consola: <https://developer.android.com/build/building-cmdline>
- Verificacion de APK: <https://developer.android.com/tools/apksigner>
- Reposo y excepciones: <https://developer.android.com/training/monitoring-device-state/doze-standby>
- Servicios en primer plano: <https://developer.android.com/develop/background-work/services/fgs/service-types>

Fuentes locales complementarias: android/README.md, tareas-moviles-lan.md, actualizacion-produccion-inventario-modular.md, usuarios-locales-y-active-directory.md y bitacora-del-proyecto.md. Los documentos historicos contienen decisiones superadas: para esta entrega, seguir el runbook fechado y verificar siempre la unidad real.

Instalacion en el trabajo sin Internet

Runbook de la entrega del 10/09/2026. Orientado a actualizar el servidor Linux existente, no a instalar todo el sistema desde cero. Ejecutar en ventana de mantenimiento con respaldo y autorizacion institucional. No se ejecutaron estos pasos contra produccion.

1. Que llevar y que se puede habilitar

Llevar output/entrega-lan.zip, generado por scripts/preparar-entrega-lan.ps1. Contiene JAR compilado, APK piloto, PDF, fuentes Markdown, V15 de referencia, bundle Git, VERSION.txt, resumen de pruebas y SHA256SUMS. No lleva base de datos, secretos, JDK, MySQL, Nginx, SDK Android ni caches de compilacion. El servidor no necesita Maven ni Android SDK para ejecutar estos binarios.

El JAR habilita web movil, visor y API. La APK incluida es debug para laboratorio: no distribuir como aplicacion productiva ni ingresar claves reales de dominio por HTTP. Si aun no hay certificado institucional y APK release, habilitar primero la web por HTTPS y realizar el piloto en un entorno de prueba con cuentas sin privilegios. No prometer aviso en reposo hasta completar la prueba de campo.

Requisitos en el trabajo: Java 21, servicio existente, MySQL accesible y respaldo verificado; DNS interno y HTTPS confiable para uso real; acceso Wi-Fi al servidor; cuentas autorizadas en TAREAS; Nginx ya instalado si se usa proxy. Si falta software, llevar paquetes y dependencias aprobados para la distribucion/arquitectura Linux. Un ZIP con el JAR no instala Java ni Nginx.

2. Inventario previo y condiciones de parada

Datos historicos del repositorio, a confirmar presencialmente:

- Aplicacion: /opt/inventario-modular.
- Unidad: inventario-modular.service. NO tocar inventario.service del sistema viejo.
- Entorno: /etc/inventario-modular/inventario-modular.env.
- ExecStart historico: Java sobre target/inventario-modular-0.0.1-SNAPSHOT.jar.
- Usuario/grupo historicos: administrador. Comprobar, no asumir.
- Perfil local: MySQL y AD configurable. Perfil casa: H2 solo para desarrollo.
- MySQL esperado: 10.15.0.62:3306/inventario_modular.
- Puerto interno: 8081. Nombre/IP del servidor de aplicacion: confirmar en el trabajo.

```
java -version
sudo systemctl show inventario-modular.service \
  -p FragmentPath -p User -p Group -p WorkingDirectory -p ExecStart
sudo systemctl cat inventario-modular.service
sudo systemctl status inventario-modular.service --no-pager
sudo ss -ltnp
timedatectl
df -h /opt
```

Inspeccionar localmente: la unidad podria contener secretos si fue configurada de otra forma. No pegar su contenido completo en chats. Comprobar hora y zona institucional: los contadores diarios usan la fecha del servidor.

PARAR si no se conoce la ruta real del JAR, hay cambios locales no comprendidos, el respaldo falla, MySQL apunta a otra base, falta TLS para credenciales reales o no hay forma de volver al JAR anterior. No sustituir

servicios del inventario viejo.

3. Transferencia y preparacion

Transportar el ZIP por el medio interno autorizado, por ejemplo USB o WinSCP. Descomprimirlo en una carpeta de entrega independiente, no sobre el checkout activo. Conservar la carpeta completa. Definir ENTREGA con la ruta real; los siguientes ejemplos suponen que esa carpeta contiene VERSION.txt y SHA256SUMS.

```
ENTREGA=/ruta/real/entrega-lan
cd "$ENTREGA"
sha256sum -c SHA256SUMS
cat VERSION.txt
```

Todos los hashes deben ser OK. Los hashes detectan cambios accidentales, no sustituyen la confianza en quien entrega el paquete. Contrastar el commit con GitLab desde la PC autorizada que lo preparo. En la LAN aislada no ejecutar git pull ni instalar dependencias.

El bundle permite continuar el codigo sin GitLab. En una copia nueva:

```
git clone -b primeros-pasos "$ENTREGA/inventario-modular.bundle" \
/ruta/de/trabajo/inventario-modular
```

Para actualizar un checkout existente, primero revisar git status y proteger cualquier cambio propio. No es obligatorio actualizar el checkout para ejecutar el JAR del kit. Si esta limpio y coincide con la historia entregada:

```
cd /opt/inventario-modular
git status --short
git bundle verify "$ENTREGA/inventario-modular.bundle"
git fetch "$ENTREGA/inventario-modular.bundle" \
refs/heads/primeros-pasos:refs/remotes/entrega/primeros-pasos
git switch primeros-pasos
git merge --ff-only refs/remotes/entrega/primeros-pasos
```

Si el merge no es fast-forward, detenerse y resolver la divergencia en desarrollo. El bundle contiene historia Git, no solo fuentes: custodiarlo como el repositorio. Su origen en un clon nuevo sera un archivo local; cambiarlo a GitLab solamente en una estacion con conectividad y autorizacion, no abrir Internet al servidor por ello.

4. Respaldo antes del primer arranque nuevo

Ensayar primero con una copia de MySQL, nunca apuntar pruebas al esquema productivo. V15 fue probada en H2; el ensayo MySQL es requisito de esta entrega. No restaurar datos H2 de casa en produccion. Durante el respaldo final, detener solo el servicio modular y coordinar que no haya otros escritores de esa misma base.

Los comandos siguientes son para una MISMA sesion bash. Sustituir JAR por la ruta comprobada en ExecStart antes de continuar. Si se pierde la sesion, recuperar la ruta exacta del respaldo; no generar otra fecha para intentar una reversion.

```
set -euo pipefail
JAR=/opt/inventario-modular/target/inventario-modular-0.0.1-SNAPSHOT.jar
test -f "$JAR"
BACKUP="/opt/backups/inventario-modular-$(date +%Y%m%d-%H%M%S)"
sudo install -d -m 700 "$BACKUP"
sudo systemctl stop inventario-modular.service
sudo cp -a "$JAR" "$BACKUP/aplicacion-anterior.jar"
sudo cp -a /etc/inventario-modular/inventario-modular.env \
"$BACKUP/entorno-anterior.env"
```

Respaldo MySQL con una cuenta autorizada para backup, no necesariamente la de la app. El comando pide la clave, no la coloca en historial. Sustituir USUARIO_BACKUP por la cuenta acordada. La redireccion se ejecuta como root y crea el archivo restringido.

```
sudo bash -c 'umask 077; mysqldump -h 10.15.0.62 -u USUARIO_BACKUP -p \
--single-transaction --routines --triggers --events \
inventario_modular > "$1/base-anterior.sql"' bash "$BACKUP"
sudo test -s "$BACKUP/base-anterior.sql"
```

Exigir salida exitosa y respaldo restaurable; un archivo no vacío por si solo no basta. Si falla, no continuar el despliegue. Resolver el respaldo o reanudar el servicio viejo de la aplicación modular para terminar la ventana sin cambios. No borrar respaldos.

5. Entorno, base y publicacion del JAR

Editar el archivo de entorno con `sudoedit`, conservando valores reales existentes. No copiar passwords desde documentacion ni activar el usuario simulado de casa. La autenticacion local de cuentas en base es distinta del acceso local simulado.

Configuracion a revisar, no pegar como reemplazo completo del archivo:

```
SPRING_PROFILES_ACTIVE=local
INVENTARIO_SERVER_PORT=8081
INVENTARIO_LAN_ONLY=true
INVENTARIO_LOCAL_AUTH_ENABLED=false
INVENTARIO_LOCAL_DB_AUTH_ENABLED=true
INVENTARIO_LDAP_ENABLED=true
INVENTARIO_DB_PRIMARY_URL=jdbc:mysql://10.15.0.62:3306/inventario_modular
INVENTARIO_DB_PRIMARY_USER=inventario_modular_app
```

INVENTARIO_DB_PRIMARY_PASSWORD y parametros LDAP deben conservar los secretos vigentes. Los alias INVENTARIO_DB_URL/USER/PASSWORD existentes tambien funcionan si no hay valores PRIMARY que los sobrescriban. LDAP requiere host, dominio, base DN y, para las consultas de directorio, la cuenta lectora configurada. Confirmar con el administrador del dominio.

El perfil local intenta fallback MySQL cuando falla el principal. No hay interruptor dedicado de fail-fast. En produccion, para evitar escribir en una base local por error, configurar INVENTARIO_DB_FALLBACK_URL/USER/PASSWORD con el MISMO destino y credenciales del principal, o acordar otra politica explicita y probarla. No escribir referencias como `${OTRA_VARIABLE}` dentro del EnvironmentFile: systemd no las expande como un shell. Si no hay conexion valida, es preferible que el servicio no inicie.

V15 se ejecuta al arrancar por `spring.sql.init.mode=always` en el perfil local. Crea dos tablas y una fila de secuencia si faltan. Requiere permisos de creacion y consulta/escritura acordados con DBA. Flyway esta deshabilitado y Hibernate usa update. NO activar Flyway: hay versiones V6 duplicadas en el historial. En otro perfil, revisar su mecanismo de migracion antes de arrancar. Ejecutar V15 manualmente solo si forma parte del procedimiento aprobado; no ejecutar todas las migraciones historicas.

Con el servicio detenido y respaldado, instalar el JAR validado preservando propietario y permisos del anterior. ENTREGA, JAR y BACKUP deben seguir definidos y comprobados.

```
sudo cp "$ENTREGA/inventario-modular.jar" "${JAR}.nuevo"
sudo chown --reference="$JAR" "${JAR}.nuevo"
sudo chmod --reference="$JAR" "${JAR}.nuevo"
sudo mv "${JAR}.nuevo" "$JAR"
sudo chown root:root /etc/inventario-modular/inventario-modular.env
sudo chmod 600 /etc/inventario-modular/inventario-modular.env
sudo systemctl start inventario-modular.service
sudo systemctl status inventario-modular.service --no-pager
sudo journalctl -u inventario-modular.service -n 120 --no-pager
curl -I http://127.0.0.1:8081/movil/login
```

No hace falta daemon-reload por cambiar solo el EnvironmentFile; si se modifica la unidad o un override, ejecutar daemon-reload antes de iniciar. Esperar a que termine el arranque. El log debe indicar conexión al MySQL principal correcto, sin fallback inesperado ni errores SQL. Con DBA, comprobar tareas_avisos y tareas_aviso_secuencia. Detener y revertir si el destino de datos o el esquema no son los esperados.

6. HTTPS institucional sin afectar otros sistemas

Usar DNS interno estable y certificado emitido por la CA institucional, con ese nombre en Subject Alternative Name. Instalar la CA pública en los celulares y en el equipo de prueba; la clave privada del servidor nunca va a teléfonos, PDF, Git o APK. El certificado debe incluir cadena, vigencia y nombre correctos. No usar curl -k ni desactivar verificación TLS como solución productiva.

NO ejecutar automáticamente scripts/setup-nginx-https.sh: usa apt-get, genera un autofirmado sin SAN y puede cambiar el sitio default compartido. La configuración histórica Nginx contiene IP/nombre de ejemplo; no copiarla sobre un sitio existente. Inventariar los virtual hosts y reservar un nombre propio, sin afectar inventario viejo.

Ejemplo de un NUEVO virtual host dedicado, a adaptar con el administrador Linux. Reemplazar tareas.intranet.ejemplo por el nombre interno real y las rutas de certificado. No activar hasta que Nginx y los certificados ya estén presentes en el servidor.

```
server {
    listen 443 ssl;
    server_name tareas.intranet.ejemplo;
    ssl_certificate /etc/ssl/certs/tareas-cadena.pem;
    ssl_certificate_key /etc/ssl/private/tareas.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    location / {
        proxy_pass http://127.0.0.1:8081;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Port $server_port;
        proxy_set_header Forwarded "";
    }
}
```

Este ejemplo supone un solo proxy: reemplaza cabeceras recibidas del cliente. Con otros proxies, definir explícitamente la cadena confiable. No exponer 8081 a clientes que puedan falsificar cabeceras forwarded. Cuando todo acceso directo y scripts de inventario ya usen el proxy, establecer SERVER_ADDRESS=127.0.0.1 y SERVER_SERVLET_SESSION_COOKIE_SECURE=true en el entorno y reiniciar el modular. Comprobar antes los clientes existentes que usen 8081 para no interrumpirlos.

```
sudo nginx -t
sudo systemctl reload nginx
curl --cacert /ruta/CA-institucional.crt \
-I https://NOMBRE_INTERNO_REAL/movil/login
```

No recargar Nginx si falla la validacion. Limitar 443 a las redes institucionales y salida a los servicios internos necesarios, sin abrir Internet. No reemplazar reglas de firewall completas desde este instructivo. Probar que Wi-Fi alcanza DNS/HTTPS; estar en el mismo segmento no garantiza acceso si hay aislamiento o ACL.

7. Instalar Android y validar el circuito

Produccion requiere una APK release firmada. El kit contiene solo piloto debug. Para un laboratorio autorizado, instalar ese piloto con cuentas de prueba; la APK acepta HTTP local, pero eso no cifra las credenciales. Preferir HTTPS tambien alli.

Cuando exista la APK autorizada, copiarla a una ruta estable legible por el usuario del servicio, por ejemplo /opt/inventario-modular/distribucion/tareas-lan.apk. Configurar en el EnvironmentFile y reiniciar el servicio:

```
INVENTARIO_MOVIL_APK_PATH=/opt/inventario-modular/distribucion/tareas-lan.apk
```

El enlace de descarga aparece en la web cuando el archivo es legible. No se descarga anonimamente: requiere login y TAREAS/VER. Tambien puede transferirse por USB/MDM interno. La APK no esta dentro del JAR ni almacenada en Git. Distribuir solo la version acordada.

1. Confirmar Android 8+ y WebView Chromium 103+. Preparar actualizaciones offline autorizadas si faltan.
2. Instalar la CA publica institucional. Instalar la APK por el mecanismo autorizado.
3. Configurar solo el origen del servidor, por ejemplo https://NOMBRE_INTERNO_REAL, sin /movil/tareas. localhost en el telefono es el telefono, no el servidor.
4. Ingresar como tecnico con TAREAS/VER y TAREAS/EDITAR. Activar Avisos y aceptar notificaciones.
5. Ajustar bateria sin restricciones segun fabricante; comprobar volumen, canal y No molestar.
6. Usar Ajustes > Probar sonido. Esperar estado Conectado antes de crear la primera tarea.
7. Desde otra cuenta administradora en PC, crear una tarea libre. Debe aparecer el aviso al siguiente ciclo nominal de consulta; registrar demora real, no prometer un plazo fijo.
8. Tocar aviso, tomar tarea, comentar y cerrarla. Verificar responsable y comentario en PC y actualizacion del visor. El autor de su propia tarea no recibe sonido.
9. Repetir con pantalla bloqueada 30 minutos, reposo, perdida/reconexion Wi-Fi, reinicio del servidor, cierre de sesion y dos tecnicos tomando la misma tarea.

Registrar modelo, Android/WebView, usuario de prueba, hora creacion/recepcion, escenario, resultado y bateria. El servicio no vuelve solo tras reiniciar el telefono o forzar cierre. Si una prueba de reposo falla, mantener piloto controlado y no anunciar despliegue general.

8. Firma release en una estacion preparada

No es necesario compilar Android en el Linux productivo. Usar una estacion con JDK, SDK y dependencias previamente preparadas. Desde android/, en Linux:

```
sh ./gradlew assembleRelease lintRelease --offline --no-daemon
```

En Windows usar gradlew.bat. --offline requiere que Gradle Wrapper, plugin y todas las dependencias ya esten en cache; no los incluye el kit. La salida release actual es unsigned. Firmar con una clave institucional

existente y custodiada, nunca con debug. Ejemplo Linux con Build Tools 35.0.0, desde android/:

```
BT="$ANDROID_HOME/build-tools/35.0.0"
"$BT/zipalign" -f -v 4 \
  app/build/outputs/apk/release/app-release-unsigned.apk \
  /ruta/segura/tareas-alineada.apk
"$BT/apksigner" sign --ks /ruta/segura/firma-institucional.jks \
  --out /ruta/segura/tareas-lan-release.apk \
  /ruta/segura/tareas-alineada.apk
"$BT/apksigner" verify --verbose --print-certs \
  /ruta/segura/tareas-lan-release.apk
```

La herramienta pide la clave: no pasarla en línea de comando. Alinear antes de firmar; no modificar el APK firmado. Registrar hash/certificado firmante y custodiar la clave para futuras actualizaciones. Incrementar versionCode en cada entrega posterior. El paquete release y el piloto tienen distinto applicationId; no son la misma app actualizada y sus sesiones/cursos no se comparten. Desactivar avisos en el piloto para evitar duplicados cuando se adopte release.

9. Aceptacion, problemas y reversion

Aceptar el servidor solo si hay login local/AD real correcto, autorizacion por roles, creacion/comentarios/toma/cierre, descarga protegida, V15 en la base correcta y HTTPS sin advertencias. Aceptar la APK solo tras firma y prueba real de fondo. Los contadores del visor tienen los pendientes descritos en el traspaso: no usarlos aun como reporte formal de productividad sin corregir la distincion entre cerradas y canceladas.

Diagnostico rapido:

- No conecta: verificar nombre DNS, puerto, Wi-Fi/ACL, listener, certificado y log Linux.
- 401: ingresar de nuevo y activar Avisos. 403: revisar permiso TAREAS y origen de red.
- No suena: probar canal, otra cuenta autora, primera sincronizacion y restricciones de bateria.
- APK no visible/404: revisar ruta configurada, archivo y lectura por el usuario systemd.
- Tabla inexistente: verificar perfil, SQL init, permisos y MySQL seleccionado; no habilitar Flyway a ciegas.
- Visor no cambia: revisar En vivo, formulario con foco o cambios sin guardar, sesion y conexion.

Para revertir binarios, usar la misma ruta BACKUP guardada y JAR verificado:

```
sudo systemctl stop inventario-modular.service
sudo cp -a "$BACKUP/aplicacion-anterior.jar" "$JAR"
sudo cp -a "$BACKUP/entorno-anterior.env" \
  /etc/inventario-modular/inventario-modular.env
sudo systemctl start inventario-modular.service
sudo systemctl status inventario-modular.service --no-pager
```

Si se modificaron unidad o Nginx, restaurar sus respaldos verificados con el administrador; este procedimiento no los modifica automaticamente. Las tablas V15 son aditivas: dejarlas en la base al volver al JAR anterior. No borrar ni restaurar MySQL a ciegas; una restauracion elimina operaciones posteriores y requiere coordinacion DBA y ventana sin escrituras. Si Hibernate altero otra estructura, evaluar compatibilidad antes de volver.

Al finalizar registrar fecha, operador, commit de VERSION.txt, hashes, respaldo usado, pruebas realizadas y excepciones. Mantener credenciales y datos de produccion fuera de GitLab.